



WEB DOMAIN TASK

Overview:

Your task is to build a **Real Estate Micro-Climate Scorer**, a web application that evaluates a given location based on environmental factors such as **air quality, walkability, and natural disaster risks**.

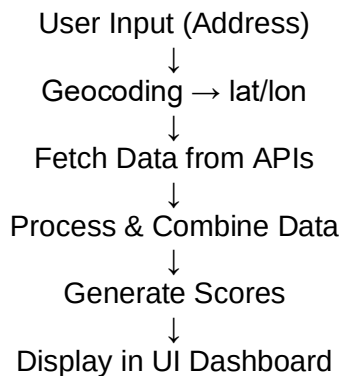
The application should integrate multiple APIs to fetch real-world data and present it through a **clear, intuitive user interface**. The goal is to help users understand important location insights that are typically not visible in standard real estate listings.

Objectives:

Core Requirements:

- Build a responsive web application using **HTML, CSS, and JavaScript** (preferably React / Next.js)
- Allow users to:
 - Enter an **address or location**
- Convert the input into **latitude and longitude coordinates**
- Fetch and display data from multiple APIs:
 - **Air Quality Data** (AQI, PM2.5)
 - **Nearby Amenities** (to estimate walkability)
 - **Natural Disaster Indicators** (earthquake and flood data)
- Display the following information:
 - AQI Score
 - PM2.5 Levels
 - Walkability Indicator (based on nearby amenities)
 - Earthquake Risk Indicator
 - Flood Risk Indicator
- Combine the above into a simple:
 - **Overall Location Score**
- Present all data in a **clean and well-structured dashboard UI**

Data Flow:



UI/UX Requirements:

- Clean and minimal interface
- Clearly separated sections for:
 - Environmental Data
 - Walkability
 - Risk Indicators
- Error handling for failed requests
- Easy to navigate

Additional Guidelines:

- **Error Handling:**
 - Handle API failures gracefully and inform the user
- **Performance:**
 - Optimize API calls where possible
- **Code Quality:**
 - Write clean, modular, and well-documented code

Evaluation Criteria:

- **Functionality:** Are all required features implemented correctly?
- **Usability:** Is the application intuitive and easy to use?
- **Design:** Is the UI clean and responsive?
- **Code Quality:** Is the code well-structured and readable?
- **Live Deployment:** The project must be hosted (Vercel / Netlify)

Bonus Features (Optional):

You may implement additional features to enhance your application. These are **not mandatory** but can improve your submission:

- User authentication (e.g., Firebase Authentication)
- Loading states while fetching data
- Responsive across devices
- Session handling (e.g., JWT-based authentication)
- Data visualization using charts (e.g., React Charts / Chart.js)
- Ability to save/bookmark locations
- Enhanced UI with animations and transitions
- **Functionality:** Are all required features implemented correctly?

Technical Stack (Suggested):

Frontend:

- React.js / Next.js
- Tailwind CSS
- Axios / Fetch

APIs:

- Air Quality API (e.g., Open-Meteo)
- Amenities Data (Overpass API / Foursquare)
- Disaster Data (USGS / NOAA)
- Geocoding API (Mapbox / OpenCage / Google)

Deployment:

- Vercel / Netlify

Submission Guidelines (Web Development Task)

1. Submit a **public GitHub repository link** containing all source code (exclude node_modules, .env) as instructed in the portal.
2. Include a **live deployed link** (Vercel / Netlify). Partial implementations are acceptable, but the work should demonstrate your best effort.
3. The repository must contain a **well-formatted README.md** with:
 - Project overview
 - Tech stack
 - Application flow (Input → Geocoding → APIs → Processing → Output)
 - Features implemented
 - Scoring logic
 - API details
 - Deployment link
 - Setup instructions
4. Ensure all features work correctly and handle API errors properly.
5. Submit via the **recruitment portal only**. Do not email submissions.
6. **Deadline:** As communicated on the recruitment portal. Late submissions will not be reviewed.

**WISHING YOU ALL THE BEST
WEB DOMAIN ADG**